Bangla Government-Service Assistant

A bilingual, source-grounded assistant for Bangladesh passport and NID services

Final report | Shaiful Islam | November 2026

Code: github.com/ShaifulPalash/bangla-government-service-assistant | Live page: bangla-gov-assistant.streamlit.app | API documentation: bangla-gov-api.onrender.com/docs

1. Problem and objectives

Information about passports and National ID (NID) cards in Bangladesh is spread across government websites, PDF forms and notices. A citizen has to find the right page, read official wording that may be in another language than the one they ask in, and judge whether it is current. A general chatbot can answer fluently but without sources, or with outdated or invented details, and a wrong fee or document list can mean a failed application or a wasted trip to an office.

This project builds an assistant that answers only from official documents, shows where every answer came from, and refuses or asks a question back when the evidence is weak. The objectives were to:

- answer in English and Bangla (including Bangla typed in English letters) from verified official sources;
- cite the source document, with its date, in every answer;
- refuse or ask for clarification when the evidence is missing, weak or out of scope;
- measure retrieval quality, answer decisions, citations and answer language on a labelled question set.

The scope is passport and NID services, covered by 17 official documents. Tax and utility services were left out for time. The assistant gives information only: it cannot submit applications or take payments, and the page says so.

2. Solution overview

A visitor uses a Streamlit chat page. The page calls a FastAPI service running in Docker. The service runs a LangGraph workflow that prepares the question, classifies it, retrieves passages, checks the evidence, asks a language model for a short cited answer and verifies the result. Everything runs on free tiers.

Layer	Technology
Page	Streamlit, hosted on Streamlit Community Cloud
API	FastAPI with Pydantic, in Docker on Render (free)
Workflow	LangGraph, with plain-Python rules for routing and safety
Retrieval	Gemini embeddings (gemini-embedding-2, 768 dimensions) and ChromaDB
Language models	Groq gpt-oss-120b first, Gemini 3.5-flash-lite as backup
Quality	pytest (372 tests), Ruff, pre-commit, GitHub Actions, docker-compose

3. AI approach and design decisions

3.1 Knowledge base

The knowledge base is 17 official documents: 6 passport pages and 11 NID documents (FAQ sections, forms, a fee schedule, and the registration rules with their amendments). Each document keeps its address and its published, updated and verified dates. Pages that need a browser were copied by hand, and scanned PDFs were read with Bangla OCR and then corrected against the originals. Mobile numbers of individual officers were removed.

3.2 Chunking and embeddings

Text is cut into passages of at most 1,000 characters with a 150-character overlap, at paragraph, line or sentence ends (including the Bangla danda), never across a page. This gives 192 passages. Each passage is embedded with Gemini and stored in ChromaDB, about 4 MB, which ships inside the Docker image. A hosted embedding model was chosen because a local model would not fit a small free server.

3.3 Retrieval

Three searches were compared on the 14 development questions that have expected documents: keyword search (BM25), meaning search, and hybrid fusion of both. Meaning search found the right document first every time (hit@1 1.00), keyword search 0.64, and the hybrid variants did worse than meaning search alone. A Bangla question shares no words with an English page, which is why word search fails across languages. A custom tokenizer was also needed, because the usual regular-expression word pattern breaks Bangla words at their vowel signs.

3.4 The workflow

1. **Prepare:** replace NID, passport and phone numbers with placeholders, run safety checks, detect the language.
2. **Classify:** passport, NID, both or unclear; refuse topics outside the scope.
3. **Retrieve:** the three best passages by meaning, limited to the service.
4. **Check the evidence:** the best match must score at least 0.755; otherwise ask which service is meant, or refuse.
5. **Answer:** a short prompt with the three passages; the model replies in JSON with citations.
6. **Verify:** every citation must point at a real passage, and every number in the answer must appear in the sources; otherwise the answer is replaced by a refusal.

The cheap steps run before any model call, so many questions cost no tokens. The evidence threshold is the midpoint between the weakest answerable development question (0.769) and the strongest weak-evidence one (0.741). Similarity scores are not absolute, so the cut-off was taken from the project's own scores.

3.5 Choosing the language model

Three free-tier models were run on ten development questions with the same passages.

Model	Errors	Invented numbers	Right language	Average time
Gemini 3.8-flash	8 of 10 (a 20-requests-a-day limit)	none, but only 2 answers	2 of 10	about 75 s

Model	Errors	Invented numbers	Right language	Average time
Gemini 3.5-flash-lite	0	2 answers	8 of 10	about 2.3 s
Groq gpt-oss-120b	0	none	7 of 10	about 1.0 s

Groq became the primary model and Gemini 3.5-flash-lite the backup. The first model was dropped. The failures that code can fix, such as answering an English question in Bangla, were fixed in code: the language is detected and stated in the prompt. The ones that cannot be fixed in code, such as invented numbers, decided the choice. The fallback also moves to the next provider when a reply cannot be read.

3.6 Keeping answers honest and cheap

Only the three retrieved passages are sent, each trimmed to 900 characters. The prompt demands the use of the sources only, exact copying of numbers, citations, and no names or contacts of individual officers. A cache answers repeated questions at once, and the model's reasoning effort is set to low. Free tiers allow only a few model calls a minute (Groq showed a limit of 8,000 tokens a minute), so these savings decide how many visitors the demo can serve.

4. Backend, frontend and engineering

Backend. FastAPI offers /health, /search, /chat and /feedback, with validated requests and clear error codes. It has an in-memory answer cache, a per-visitor rate limit, a global ceiling on uncached questions, and JSON logs. A small SQLite log stores outcomes and timings but never the question text. Timeouts and daily-limit errors fail fast to the backup model.

Frontend. The Streamlit page has a banner, a language toggle for the interface, example questions, source cards with the document title, date and link, a feedback form, and a privacy notice. In Bangla answers the page converts digits and taka amounts to Bangla without changing any value. It wakes the sleeping API on the first visit and explains the wait.

Deployment. The API runs from a separate private deploy repository that contains the prebuilt index, on Render's free tier. The page runs on Streamlit Community Cloud. A smoke-test script checks any running instance: health, a cited answer, the cache, clarification, refusal, personal-number removal, validation, feedback and the rate limit.

Engineering practice. The work was done on short branches merged with merge commits and tagged at milestones. There are 372 unit tests. Ruff and pre-commit hooks (including a check that blocks API-key-like strings) run before each commit, and GitHub Actions runs lint and tests on every push. A docker-compose file starts the whole stack with one command.

5. Results

5.1 Test set and method

There are 30 labelled questions in English, Bangla and Bangla in English letters: 20 answerable, 5 out of scope, 4 ambiguous and 1 about outdated information. Twenty were used to build and tune the system. Ten were kept unseen and run once at the end, against targets that were committed before the run. The test labels were verified against the documents, not against system output.

5.2 Final test results (10 questions, run once)

Metric	Target	Result	Met
Decision accuracy	0.80	0.90	yes
Refusal recall	0.90	1.00	yes
Refusal precision	0.80	0.67	no
Citation accuracy	0.80	0.83	yes
Answer language accuracy	0.90	1.00	yes

On the development set all 20 questions ended in the labelled outcome, but these scores are optimistic because the system was tuned on them. One question moves a test score by 10 points, so the numbers show large problems and cannot prove a precise level.

5.3 What went wrong

- **A wrong refusal (q015).** The right passage ranked first but scored 0.725, below the threshold, because an English question was matched with a Bangla source. Lowering the threshold would not help: a question that should ask for clarification scored 0.737.
- **A wrong figure (q030).** The model took a price printed on a scanned Gazette page for a fee. The number was in the source, so the number check passed it. Reading the six answered test questions by hand against their sources gave four supported, one partly correct and one wrong.

5.4 Fixes after the test run

The Gazette price notes were removed from the scanned text. A manual test then showed passport fee tables for different applicant groups mixed in one answer (taka and dollar amounts shown as alternatives), because the tables share identical row labels and a chunk boundary can separate a table from its heading. Every fee row now carries its applicant group, fee questions are searched as inside Bangladesh by default, and a prompt rule defines abbreviations such as GD (General Diary, a police report). The development set stayed at 20 of 20. The test set was not re-run, so section 5.2 describes the system before these fixes.

5.5 Operation

The deployed API passed all 10 smoke checks after the final deploy. A fresh answer took about 1.5 to 2 seconds and a repeated question about 0.1 seconds. A local container used 111 MiB of memory after a full smoke test; Render's own memory chart was not read. Free hosting sleeps when idle, and during testing one cold start did not complete and the service had to be restarted from the dashboard.

6. Limitations

- Only passport and NID are covered, from 17 documents; tax and utilities are not.
- Rules and fees change; every answer shows its source date.
- Renewal and re-issue fees are not stated in the collected documents, so those questions are declined.
- The model chooses its citations and can add a detail that no source states (for example an invented form name). The number check does not catch words or single-digit numbers.
- The evidence threshold was tuned on 20 questions and the margin is thin.
- Only ten test questions, one reader for answer quality, and no test with many visitors at once.

- The scanned legal texts were only partly checked against the originals, and the reuse terms of the source websites were not reviewed, so the documents and the index are not redistributed in the public repository.

7. Conclusion and future work

The project delivers a working, deployed, bilingual assistant whose answers are grounded in official documents, cited with dates, and checked by code before they reach the user. The decisions came from measurements on the project's own questions: meaning search over hybrid, a rank of failures by whether code can fix them, and a threshold taken from data. The final test was honest about its misses, and the fixes that followed are labelled as made after it.

Next steps would be sentence-level checking of answers against their sources, a larger labelled set graded by more than one reader, a lower evidence threshold when the service is named, and more documents (tax, utilities, and renewal fees if an official page states them).

References

- Bangladesh e-Passport Portal: epassport.gov.bd (instructions, passport fees, application form).
- Department of Immigration and Passports, citizen charter: dip.gov.bd.
- NID services portal and FAQ: services.nidw.gov.bd/nid-pub (including the fee schedule).
- Bangladesh Election Commission gazette notices on NID registration and data regulations (see data/sources.yaml in the repository for each address and date).